

State Backends & Azure Auth

Move state off your laptop into shared, locked cloud storage — and meet the ways Terraform authenticates to Azure.

Remote state

Backends

Azure Storage backend

Azure auth

Lab 05

Today's roadmap



1 - Why remote state

Local state breaks the moment a team is involved.



2 - Backends

Where state lives, and how to create the storage.



3 - Azure auth

CLI, service principal, managed identity, OIDC.



4 - Lab 05

Store your project's state in an Azure Storage Account.

01

Managing State at Scale

The single most important decision before you use Terraform for real.

Local state doesn't scale

Every run so far created a **local** `terraform.tfstate`. Fine for learning — a liability in production.

- Only you have it — teammates can't collaborate
- No locking → two applies corrupt each other
- Lose the laptop, lose track of your infra
- State holds **secrets** — it needs protection

⚠ Store state in a **centralized, secure, locked** place — plan this before your first real project.

Terraform Backend

A **backend** defines where state is stored. Default is `local`; real projects use a cloud backend.

- Azure Storage, AWS S3, GCS, Kubernetes...
- Or HCP Terraform (HashiCorp Cloud)
- Configured in a `backend` block

HCL

```
terraform {  
  backend "azurerm" {  
    resource_group_name = "tfstate-rg"  
    storage_account_name = "tbernardotfstate"  
    container_name       = "tfstate"  
    key                   = "app.tfstate"  
  }  
}
```

BASH

```
terraform init -migrate-state # move local state to Azure
```

How to create the backend storage



Manually

Portal or CLI script. Easy & fast — but not repeatable and error-prone.



Other IaC

ARM/Bicep/CLI in your pipeline. Repeatable — but another tech to learn.



Local, then upload

Apply locally, then migrate state to the new storage. Pure Terraform.



The "local then migrate" flow is exactly what today's demo & lab walk through.

Azure Storage as a backend

- State stored as a **blob** in a container
- Native **state locking** & consistency checks
- Auth options: Access Key, Azure AD, SAS token

✓ **Preferred:** Azure AD + OIDC (workload identity federation) — no long-lived keys.

```
use_azuread_auth = true , use_oidc = true
```

02

Authenticating to Azure

Four mechanisms — pick by where Terraform runs.

Azure authentication methods



Azure CLI

Uses your `az login` session. Ideal for local dev — not for pipelines.



Service Principal

Client ID + secret + tenant. Automatable, RBAC-scoped — but you manage the secret.



Managed Identity

No secrets — but only when the runner lives in Azure (VM, AKS...).



Workload Identity Federation

Secretless OIDC from GitHub Actions & others. The modern CI/CD choice.

ARM_environment variables

In pipelines, feed Terraform its Azure settings through `ARM_*` environment variables — no secrets in code.



Same pattern the `azurerm` backend uses for its own auth.

BASH

```
export ARM_CLIENT_ID="..."
export ARM_SUBSCRIPTION_ID="..."
export ARM_TENANT_ID="..."
export ARM_USE_OIDC="true"
```

 LIVE DEMO

Migrate local state to Azure

Create a storage account, add the `azurerm` backend block, and run `init` to migrate existing state into the cloud.

Demo · 05 **Provider** · azurerm **Auth** · az CLI

Store your state in Azure

Provision an Azure Storage Account, point your project's backend at it, and migrate the state — then push the code to your course repo.

Shared subscription: use your assigned `<prefix>` and one shared storage account with a per-student state `key`.

Key takeaways

✓ Remote state is mandatory

Shared, locked, secure — plan it first.

✓ azurerm backend

State as a locked blob in Azure Storage.

✓ Auth fits the runner

CLI locally, OIDC in CI/CD.

✓ Next up

Session 06 — building real Azure infrastructure.



Questions?

Up next — [Session 06: Building Azure Infrastructure.](#)